

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE NAME: Artificial Intelligence**

**COURSE CODE: CSA17**

---

### COURSE OUTCOME COVERED

**CO5:** *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

*Assessment Tool Weightage: 20%*

---

**QUESTIONS: 5**

**Duration : 1 Hour**  
**Total Marks:50**

Annexure A

### Reverse Engineering

---

#### **Code of Conduct:**

*I \_\_\_\_\_ K. Deeksha Sree(192424097)\_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student :Deeksha Sree**

Annexure A

## Reverse Engineering

**1. Reverse Engineering a Recommendation System**

A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

**2. Reverse Engineering a Smart Traffic Management System**

A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

**3. Reverse Engineering a Fraud Detection Pipeline**

A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

**4. Reverse Engineering a Personalized Learning System**

An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

**5. Reverse Engineering a Cloud-Based AI Monitoring System**

A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

## RUBRICS FOR CALCULATION

| Criteria                            | Weightage | Level 4 – Exemplary                                                          | Level 3 – Proficient                                 | Level 2 – Developing                           | Level 1 – Beginning                               |
|-------------------------------------|-----------|------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------|---------------------------------------------------|
| Understanding of Industry Problem   | 20%       | Comprehensive understanding of real-world problem and constraints            | Good understanding with minor gaps in requirements   | Basic understanding; some requirements unclear | Poor understanding; misinterprets problem context |
| Application of Engineering Concepts | 25%       | Applies advanced and relevant concepts effectively to solve the problem      | Applies appropriate concepts with minor errors       | Limited application of concepts                | Incorrect or no application of concepts           |
| Solution Design                     | 25%       | Innovative, practical, and feasible solution aligned with industry standards | Logical and feasible solution with minor gaps        | Basic solution with limited feasibility        | Unclear or impractical solution                   |
| Use of Tools                        | 15%       | Effective use of modern tools, technologies, and real data                   | Appropriate use of tools with correct implementation | Limited or inconsistent use of tools           | Minimal or incorrect use of tools                 |
| Analysis                            | 10%       | Thorough analysis with validated results and performance evaluation          | Good analysis with reasonable validation             | Basic analysis with limited validation         | Poor or no analysis                               |
| Professionalism                     | 5%        | Highly professional, well-documented, and clearly presented work             | Good documentation and presentation                  | Basic documentation with some gaps             | Poor documentation and presentation               |

## 1. Reverse Engineering a Recommendation System

### Architecture:

The e-commerce recommendation system collects user activity such as searches, clicks, product views, purchases, ratings, and cart activity. The data is stored in distributed storage and processed using PySpark.

### RDD-based processing:

- `textFile()` reads large-scale user/product data.
- `map()` converts raw records into (user, product) pairs.
- `filter()` removes invalid or incomplete records.
- `groupByKey()` or `reduceByKey()` groups user interactions.
- `join()` combines user, product, and rating information.
- The resulting RDD is used to calculate product similarity and recommendation scores.

### Intelligent

agents:

Recommendation agents continuously monitor user behavior. When a user searches for or purchases a product, the agent updates the user's preference profile and generates new recommendations.

### Output:

The system dynamically displays personalized products based on the user's recent and historical behavior.

## 2. Reverse Engineering a Smart Traffic Management System

### Architecture:

The system receives real-time information from cameras, IoT sensors, GPS devices, and traffic signals. This information is processed continuously to estimate vehicle density and congestion.

### RDD-based processing:

- Traffic sensor data is collected as streaming records.
- `map()` extracts location, vehicle count, and timestamp.
- `filter()` removes incorrect sensor readings.
- `reduceByKey()` calculates vehicle density for each road.
- `groupByKey()` groups traffic information by junction.
- The processed data is analyzed to identify congested roads.

### Intelligent

agents:

Traffic-control agents monitor individual junctions. They communicate with neighboring agents to understand traffic conditions and coordinate signal timings.

Decision

making:

If vehicle density increases on one road, the agent can increase the green-light duration. Since traffic decisions must be made quickly, only recent data is processed.

Output:

Traffic signals automatically change to reduce congestion and improve traffic flow.

### 3. Reverse Engineering a Fraud Detection Pipeline

Architecture:

The banking system receives transaction information such as amount, location, time, merchant, device, and customer history. The transaction is processed through a distributed fraud-detection pipeline.

RDD-based processing:

- `textFile()` or streaming input receives transaction data.
- `map()` extracts important transaction attributes.
- `filter()` removes invalid transactions.
- `map()` performs feature extraction, such as transaction frequency and unusual amount.
- `reduceByKey()` calculates customer-level transaction statistics.
- The processed RDD is passed to an anomaly-detection or machine-learning model.

Intelligent

agents:

A fraud-detection agent evaluates each transaction. If suspicious behavior is detected, an alert agent can immediately notify the bank or temporarily block the transaction.

Example:

If a customer normally spends ₹1,000–₹5,000 but suddenly makes a ₹2,00,000 transaction from an unusual location, the system may assign a high fraud score.

Output:

The system generates a legitimate/fraudulent decision or risk score within milliseconds.

### 4. Reverse Engineering a Personalized Learning System

Architecture:

The platform collects student information such as quiz scores, assignment marks, video-watching time, number of attempts, and learning activity.

RDD-based processing:

- `textFile()` loads learner activity data.
- `map()` converts raw activity into student-performance records.
- `filter()` removes invalid records.

- `groupByKey()` groups activities by student.
- `reduceByKey()` calculates performance statistics.
- `join()` combines performance with course and content information.

Intelligentagents:

A learner agent maintains an individual student profile. It continuously observes performance and determines whether the student needs easier, normal, or advanced content.

Example:

If a student repeatedly scores below 50%, the agent can recommend easier lessons and additional practice. If the student consistently scores above 90%, it can recommend advanced material.

Output:

The system dynamically changes the difficulty of learning materials and provides personalized feedback.

## 5. Reverse Engineering a Cloud-Based AI Monitoring System

Architecture:

The cloud system collects CPU usage, memory usage, network traffic, application logs, errors, and service-health information from multiple servers.

RDD-based processing:

- `textFile()` loads large-scale distributed log files.
- `map()` extracts timestamp, server ID, resource usage, and error information.
- `filter()` identifies important events and errors.
- `groupByKey()` groups logs by server or service.
- `reduceByKey()` calculates metrics such as average CPU usage.
- RDD lineage provides fault tolerance, allowing lost partitions to be recomputed.

Intelligentagents:

Different agents perform different tasks:

- Monitoring agent: observes system health.
- Anomaly-detection agent: identifies unusual resource usage.
- Scaling agent: increases or decreases cloud resources.
- Response agent: handles failures and alerts administrators.

Example:

If CPU usage continuously exceeds 90%, the monitoring agent detects the problem. The scaling agent can automatically add resources, while the response agent records the event and sends an alert.

Output:

The system automatically detects failures, scales resources, and maintains reliable cloud services.

## Quick comparison

| System           | Main Data         | RDD Operations                      | Intelligent Agent        | Main Decision          |
|------------------|-------------------|-------------------------------------|--------------------------|------------------------|
| Recommendation   | Clicks, purchases | searches, Map, Filter, Join, Reduce | Recommendation Agent     | Which products to show |
| Traffic          | Sensors, GPS      | cameras, Map, Filter, Reduce        | Traffic Agent            | Signal timing          |
| Fraud            | Transactions      | Map, Filter, Reduce, Join           | Fraud/Alert Agent        | Fraud or genuine       |
| Learning         | Scores, activity  | Map, Group, Reduce, Join            | Learner Agent            | Content difficulty     |
| Cloud Monitoring | Logs, CPU, memory | Map, Filter, Reduce, Group          | Monitoring/Scaling Agent | Scale or respond       |

Common reverse-engineering flow:

Data Sources → Data Collection → PySpark RDD → Transformations → Analysis/ML → Intelligent Agents → Decision → Real-Time Action